



Claude Code 핸드북

seobuk-kim framework

v1.0 — Full-Stack Development Framework

Seobuk Corp. | 2026-04-16 | 14 Chapters

목차

입문

빠른 시작

기본 사용법

단축키 & 셸 설정

중급

Settings 최적화

스킬 & 커맨드

Git 워크플로우

Playwright 자동화

MCP 서버

고급

멀티에이전트

cmux 멀티세션

컨텍스트 엔지니어링

CI/CD 셋업

부록

문제 해결

처음 시작하기 — 설치부터 첫 대화까지

Claude Code가 뭔가요?

Claude Code는 AI 비서가 내 컴퓨터에서 직접 일하는 도구입니다.

구분	일반 CHATGPT	CLAUDE CODE
어디서?	웹 브라우저에서만	내 컴퓨터에서 직접
뭘 할 수 있나?	물어보고 답 받기	파일을 직접 읽고 수정, 명령어 실행
파일 수정	불가능	직접 가능
프로젝트 이해도	낮음 (파일을 못 봄)	높음 (전체 파일 분석)

쉽게 말해: "ChatGPT가 내 컴퓨터에 눈과 손을 가진 셈"이라고 생각하면 됩니다.

이 핸드북은 누구를 위한 건가요?

- 코드를 직접 쓰지는 않지만, AI에게 일을 시키는 분
- 개발 용어를 잘 모르지만, AI를 도구로 활용하고 싶은 분
- "방향만 잡아주면 나머지는 AI가 다 합니다"

💡 이 핸드북의 핵심 원칙: 당신은 방향만 잡으세요. 코드, 설정, 명령어는 AI가 다 합니다.

1단계: 설치하기 (AI한테 시킬 수 없는 유일한 단계)

설치만큼은 직접 해야 합니다. 이후의 모든 작업은 AI한테 자연어로 시킬 수 있습니다.

Mac에서 설치

- 터미널(컴퓨터에 글자로 명령하는 검은 화면) 열기
 - 맥 오른쪽 위 검색 아이콘(돋보기) 클릭 → "터미널" 입력 → 엔터
- 아래 한 줄을 복사해서 터미널에 붙여넣고 엔터

```
brew install anthropic/claude/claude
```

❌ **brew: command not found** 에러가 나면? → Homebrew(맥용 프로그램 설치 도구)가 없는 겁니다. 아래를 먼저 터미널에 붙여넣고 엔터:

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/
```

설치 완료 후 터미널을 닫았다 다시 열고, **brew install anthropic/claude/claude** 를 재시도하세요.

Homebrew가 계속 안 되면 이 방법을 시도:

```
npm install -g @anthropic-ai/claude
```

❌ **npm: command not found** 이면? → Node.js를 먼저 설치해야 합니다. <https://nodejs.org> 에서 LTS 버전을 다운로드해 설치한 뒤 재시도하세요.

설치 확인:

```
claude --version
```

✅ **Claude Code v0.x** 같은 버전 정보가 나오면 설치 완료!

Windows에서 설치

명령 프롬프트 열기 — 시작 메뉴 → "cmd" 입력 → 엔터

아래 명령어 복사-붙여넣기:

```
npm install -g @anthropic-ai/claude
```

❌ **npm을 찾을 수 없음** 에러가 나면? → <https://nodejs.org> 에서 LTS 버전 설치 후 명령 프롬프트를 닫았다 다시 열고 재시도

2단계: 처음 실행하고 로그인하기

터미널에서 `claude` 입력 후 엔터

`/login` 입력 후 엔터 → 자동으로 브라우저가 열림

Anthropic 계정으로 로그인 (없으면 가입)

"승인" 버튼 클릭

터미널로 돌아오면 완료!

💡 로그인은 처음 한 번만 하면 됩니다. 이후에는 자동으로 로그인 상태가 유지됩니다.

프로젝트 폴더에서 시작하기

Claude Code를 쓸 때는 항상 프로젝트 폴더에서 시작해야 합니다.

```
cd ~/내_프로젝트_폴더_경로
claude
```

💡 `cd` 는 "이 폴더로 이동해"라는 뜻입니다. 폴더는 파일들이 들어있는 상자 같은 것입니다.

? 어떤 폴더에서 시작해야 하는지 모르겠어요. → 개발팀에게 "프로젝트 폴더 경로가 뭐예요?"라고 물어보세요.

⚠️ 잘못된 폴더에서 시작하면? → AI가 엉뚱한 파일을 읽게 됩니다. 반드시 프로젝트 폴더에서 시작하세요.

3단계: 첫 대화 — 직접 해보기

설치가 끝났으면 바로 실습해봅시다!

실습 1: 프로젝트 이해하기

💬 AI한테 이렇게 말하세요:

"이 프로젝트 구조 설명해줘"

AI가 폴더 안의 파일들을 보고 자동으로 설명해줍니다.

어떤 프레임워크를 쓰는지

주요 폴더가 뭘 담당하는지

중요한 설정 파일이 뭔지

실습 2: 파일 찾기

💬 AI한테 이렇게 말하세요:

"로그인 관련 파일이 어디에 있어?"

AI가 전체 프로젝트를 검색해서 관련 파일을 찾아줍니다.

실습 3: 에러 이해하기

💬 AI한테 이렇게 말하세요:

"이 에러 메시지가 뭔 의미야? TypeError: undefined is not a function"

AI가 에러의 의미를 쉽게 풀어서 설명하고, 해결 방법을 알려줍니다.

실습 4: 코드 수정 시키기

💬 AI한테 이렇게 말하세요:

"프로필 사진 업로드 기능 만들어줘"

AI가 코드를 작성하고 "이거 할까요?"라고 물어봅니다.

y 입력 → 적용

n 입력 → 거절

💡 처음에는 AI가 뭘 하려는지 하나씩 확인하면서 진행하세요. 익숙해지면 자동 모드(03장 참고)로 바꿀 수 있습니다.

4단계: 알아두면 좋은 것들

모드

처음에는 **default 모드**(매번 승인을 물어보는 모드)로 시작됩니다. 이것만으로 충분합니다. 익숙해진 뒤 모드를 바꾸고 싶다면 03장을 참고하세요.

자주 쓰는 명령어

대화 중에 아래 명령어들을 쓸 수 있습니다.

명령어	뜻	언제 쓰나
/help	사용 가능한 명령어 목록 보기	뭘 할 수 있는지 모를 때

명령어	뜻	언제 쓰나
<code>/clear</code>	대화 전부 지우고 새로 시작	한 작업 끝나고 새 작업 시작할 때
<code>/compact</code>	대화를 요약해서 기억력 정리	AI가 느려질 때
<code>/status</code>	지금 상태 확인	현재 설정, 비용 확인할 때
<code>/exit</code>	Claude Code 종료	일 끝낼 때 (또는 Ctrl + C)

/clear와 /compact의 차이

이 두 명령어는 자주 쓰게 되니 차이를 꼭 알아두세요.

구분	<code>/CLEAR</code>	<code>/COMPACT</code>
하는 일	대화 전부 삭제	대화를 요약해서 압축
기억력	100%로 복구	50~70%로 줄어듦
진행상황	사라짐	요약으로 유지
언제 쓰나	새 주제로 넘어갈 때	같은 주제를 계속할 때

💡 헷갈리면 이렇게 생각하세요:

`/clear` = "새 시작" (기억 전부 삭제)

`/compact` = "정리하고 계속" (기억 요약 유지)

💡 꼭 기억하세요

- ✅ Claude Code 실행 전에 반드시 프로젝트 폴더로 이동 (`cd`)
- ✅ 파일 경로는 정확하게 (예: `src/components/Button.js`)
- ✅ 한 번에 한 가지만 시키기 — AI가 더 집중합니다
- ✅ 작업이 끝나면 `/clear` 로 초기화
- ✅ AI 응답이 느리면 `/compact`
- ✅ 모르는 건 AI한테 물어보세요 — "이게 뭐야?"라고 하면 설명해줍니다

다음 단계

01장 — 상황별 AI 활용법에서 더 다양한 활용법을 배울 수 있습니다.

이런 걸 시킬 수 있어요 — 상황별 AI 활용법

Claude Code를 어떻게 쓸지 모르겠다면? 7가지 상황별로 정리했습니다.

💡 모든 예시에서 AI한테 자연어로 말하면 됩니다. 명령어를 외울 필요 없습니다.

상황 1: 코드가 이해 안 될 때

이런 상황일 때

다른 사람이 쓴 코드를 봐야 하는데 뭔 말인지 모를 때

개발팀이 "이 파일 확인해주세요"라고 했는데 뭔 뜻인지 모를 때

💬 AI한테 이렇게 말하세요:

"이 파일이 뭐 하는 건지 쉽게 설명해줘 src/api/users.js"

"이 함수 왜 있는 거야? src/utils/formatDate.js의 format() 함수"

"45번 줄 설명해줄래?"

AI가 파일을 읽고, 쉬운 말로 설명하고, 왜 필요한지 알려줍니다.

💡 "이해 못한 척" 물어보면 AI가 더 쉽게 설명합니다. "나 코드 모르는데, 이걸 5살한테 설명하듯 설명해줘"라고 해보세요.

상황 2: 뭔가 고치고 싶을 때 (버그 찾기)

이런 상황일 때

"로그인이 안 돼요"라는 보고를 받았을 때

에러 메시지가 나오는데 뭔 뜻인지 모를 때

화면에 뭔가가 안 보이거나 이상하게 보일 때

💬 AI한테 이렇게 말하세요:

"로그인이 안 되는데 버그를 찾아줘 src/auth/login.js"

"이 에러가 뭘 의미야? TypeError: Cannot read properties of undefined"

"프로필 이미지가 안 뜨는데 src/components/Profile.tsx 검토해줘"

AI가 코드를 분석하고, 문제가 뭔지 정확하게 설명하고, 고친 코드를 제시합니다. default 모드에서는 승인 여부를 물어봅니다 — **y** 를 입력하면 적용, **n** 이면 거절.

💡 에러 메시지를 그대로 복사해서 붙여넣으면 AI가 훨씬 빠르게 원인을 찾습니다.

상황 3: 새로운 기능을 만들고 싶을 때

이런 상황일 때

기획서에 새 기능이 추가됐을 때

"이런 기능 있으면 좋겠다"라고 생각했을 때

💬 AI한테 이렇게 말하세요:

"회원가입 페이지에 비밀번호 확인 입력창 추가해줘"

"사용자 목록을 이름으로 검색하는 기능 만들어줘. 백엔드는 src/models/User.js"

"다크 모드 기능 추가해줘. 지금 있는 테마 설정 참고해서"

AI가 기존 코드 패턴을 분석한 뒤 새 코드를 작성하고, 기존 코드와 어떻게 연결되는지 설명합니다.

💡 원하는 결과를 구체적으로 말할수록 AI가 정확하게 만듭니다.

❌ "회원가입 고쳐줘" (뭘 고치라는 건지 모호)

✅ "회원가입 페이지에서 비밀번호 8자 이상, 특수문자 포함 규칙 추가해줘"

상황 4: 파일을 찾고 싶을 때

이런 상황일 때

프로젝트에서 특정 기능이 어디에 구현되어 있는지 모를 때

특정 텍스트나 설정이 어디에 있는지 찾고 싶을 때

💬 AI한테 이렇게 말하세요:

"로그인 관련 파일이 어디에 있어? 전체 프로젝트에서 찾아줘"

"API_KEY라는 상수가 어디서 정의되어 있어?"

"TODO 주석을 모두 찾아줘"

"모든 테스트 파일을 찾아줘"

AI가 전체 프로젝트를 검색해서 매칭되는 파일을 나열하고 각각 뭐 하는 건지 설명합니다.

상황 5: 웹에서 뭔가 찾고 싶을 때

이런 상황일 때

라이브러리의 최신 사용법을 알고 싶을 때

에러 메시지를 검색하고 싶을 때

기술 관련 최신 뉴스를 알고 싶을 때

💬 AI한테 이렇게 말하세요:

"React 최신 버전이 뭐야? 어떤 새로운 기능이 추가됐어?"

"CORS 에러 해결 방법 찾아줘"

"Node.js에서 파일 업로드하는 가장 좋은 라이브러리가 뭐야?"

"이 에러가 뭘 의미인지 웹에서 찾아줘: socket hang up"

AI가 인터넷을 검색하고 신뢰할 수 있는 최신 정보를 정리해줍니다.

💡 웹 검색은 MCP(07장 참고)가 연결되어 있을 때 가능합니다.

상황 6: 코드를 정리하고 싶을 때

이런 상황일 때

코드가 복잡해서 나중에 보면 이해가 안 될 것 같을 때

중복 코드가 많을 때

설명 주석을 달고 싶을 때

💬 AI한테 이렇게 말하세요:

"이 코드 좀 복잡한데 깔끔하게 정리해줘 src/utils/processing.js"

"중복된 코드를 정리해줘 src/components/ 폴더"

"각 함수에 설명하는 주석 달아줘"

"TypeScript 타입을 추가해줘 src/api.js"

AI가 코드를 분석하고 왜 이렇게 변경했는지 설명하면서 개선된 버전을 제시합니다. 적용 전에 승인을 물어봅니다.

상황 7: 테스트하고 싶을 때

이런 상황일 때

코드를 수정한 뒤 잘 작동하는지 확인하고 싶을 때

배포 전에 문제가 없는지 확인하고 싶을 때

💬 AI한테 이렇게 말하세요:

"테스트 돌려줘"

"이 함수가 잘 작동하는지 확인해봐 src/utils/math.js"

"프로젝트를 빌드해서 에러가 있는지 확인해줘"

"변경사항이 잘 적용됐는지 확인해줘"

AI가 실제로 명령어를 실행하고 결과를 보여줍니다. 에러가 있으면 원인을 설명하고 해결 방법도 제시합니다.

보너스: 이런 것도 시킬 수 있어요

문서 작성

💬 AI한테 이렇게 말하세요:

"이 프로젝트의 API 문서를 만들어줘"

"README.md를 쉽게 다시 써줘"

번역

💬 AI한테 이렇게 말하세요:

"이 에러 메시지를 한국어로 번역해줘"

"이 영문 문서를 한국어로 요약해줘"

비교 분석

💬 AI한테 이렇게 말하세요:

"이 두 파일의 차이점이 뭐야? src/old.js vs src/new.js"

"이 라이브러리와 저 라이브러리 중 뭐가 더 좋아?"

💡 꿀팁 3가지

팁 1: 정확한 파일 경로 쓰기

❌ "이 파일 확인해줘. utils에서 뭔가"

✅ "이 파일 확인해줘 src/utils/auth.js"

→ AI가 정확한 파일을 바로 찾아서 더 빠릅니다.

💡 파일 경로를 모르겠으면 "로그인 관련 파일 어디야?"라고 먼저 물어보세요.

팁 2: 모르면 솔직하게

❌ "이걸 최적화해줘"

✅ "이 코드가 뭐 하는 건지 이해가 안 가. 쉽게 설명해줘"

→ AI가 더 쉬운 말로 설명합니다. 모르는 걸 물어보는 게 더 좋은 결과를 만듭니다.

팁 3: 한 번에 한 가지만

❌ "로그인 버그도 고쳐주고, 회원가입도 추가하고, 비밀번호 초기화도 만들어줘"

✅ "먼저 로그인 버그를 찾아줘" → 끝나면 → "다음으로 회원가입 추가해줘"

→ AI가 더 집중하고 실수도 줄어듭니다. 한 작업이 끝나면 `/clear` 로 정리하세요. (00장 참고)

다음 단계

02장 — 더 빠르게 쓰는 법 — 단축키와 편의 설정

03장 — 내 취향대로 설정하기 — 환경 설정

더 빠르게 쓰는 법 — 단축키와 편의 설정

Claude Code를 더 빨리 쓰려면 몇 가지 단축키와 편의 기능을 알아두면 좋습니다.

꼭 알아야 할 단축키 5개

이 단축키들은 거의 모든 터미널(글자로 명령하는 검은 화면)에서 같습니다.

단축키	뜻	언제 쓰나
Escape	지금 입력한 것 취소	작성 중인 질문이 마음에 안 들 때
Ctrl + C	AI의 답변 중단	답변이 너무 길어질 때, 뭔가 잘못됐을 때
Ctrl + D	Claude Code 종료	일을 다 끝낼 때 (/exit와 같음)
Tab	파일 경로 자동 완성	src/com 입력 후 Tab → src/components/
↑ 화살표	이전 질문 다시 불러오기	같은 질문을 또 하고 싶을 때

단축키 사용 예시

AI가 너무 길게 답변할 때:

Ctrl + C → AI가 즉시 멈춤

더 간결하게 다시 질문: "짧게 요약해서 답해줘"

파일 경로를 자동으로 채우고 싶을 때:

src/com 입력 → **Tab** 누르기

src/components/ 로 자동 완성

계속 입력해서 src/components/Bu → **Tab** → src/components/Button.tsx

이전에 했던 질문을 다시 하고 싶을 때:

↑ 화살표를 눌러서 이전 질문 찾기

수정해서 다시 보내거나 그대로 엔터

💡 이 3개만 기억하면 됩니다: **Escape** (취소), **Ctrl+C** (중단), **Tab** (자동완성)

자주 쓰는 명령어 줄이기 (alias)

이런 상황일 때

매번 `claude` 를 치는 것도 귀찮을 때. 또는 `claude --resume latest` 같은 긴 명령어를 반복할 때. **alias**(별칭)란 자주 쓰는 명령어를 짧게 줄인 것입니다. 예를 들어 `claude` 를 `cc` 로 줄일 수 있습니다.

💬 AI한테 이렇게 말하세요:

"claude를 cc로, claude --mode plan을 ccp로, claude --resume latest를 ccr로 alias 설정해 줘"

AI가 터미널 설정 파일(`.zshrc` 또는 `.bashrc`)을 자동으로 수정합니다.

설정 후에는 이렇게 쓸 수 있습니다

짧은 명령어	원래 명령어	설명
<code>cc</code>	<code>claude</code>	Claude Code 시작
<code>ccp</code>	<code>claude --mode plan</code>	분석 전용 모드 (파일 수정 안 함)
<code>ccr</code>	<code>claude --resume latest</code>	이전 대화 이어가기

더 많은 alias 아이디어

💬 AI한테 이렇게 말하세요:

"자주 쓸 만한 Claude Code alias를 추천해서 설정해줘"

AI가 당신의 사용 패턴에 맞는 alias를 추천합니다.

💡 Windows에서도 설정할 수 있지만 방식이 조금 다릅니다. AI한테 "Windows에서 alias 설정해줘"라고 하면 PowerShell 프로파일을 수정해줍니다.

여러 작업을 동시에 하고 싶을 때

이런 상황일 때

왼쪽 창에서는 AI가 코드 수정 중이고, 오른쪽 창에서는 테스트를 돌리고 싶을 때.

💬 AI한테 이렇게 말하세요:

"작업 환경 세팅해줘. 터미널 창을 나눠서 쓰고 싶어"

AI가 tmux(터미널 창 분할 도구 — 브라우저 탭처럼 터미널 창을 여러 개 띄우는 것) 설치와 설정을 자동으로 처리합니다.

설치 후 활용 예시

왼쪽 창	오른쪽 창
Claude Code에서 코드 수정	터미널에서 테스트 실행
기능 개발	로그 모니터링
버그 수정	브라우저 확인

자세한 내용은 09장 — 멀티세션을 참고하세요.

와이파이가 바뀌어도 끊기지 않는 원격 접속

이런 상황일 때

카페에서 회사로 이동할 때, 와이파이가 바뀌면서 원격 접속(서버 접속 등)이 끊어질 때.

💬 AI한테 이렇게 말하세요:

"mosh 설치하고 원격 접속 설정해줘"

AI가 mosh(Mobile Shell — 와이파이가 바뀌어도 자동 재연결되는 원격 접속 도구) 설치와 설정을 처리합니다.

mosh가 좋은 이유

구분	일반 SSH	MOSH
와이파이 변경	연결 끊김	자동 재연결
지하철 이동	끊김 반복	자동 복구
노트북 덮개 닫기	연결 끊김	재연결

⚠ 서버 쪽에도 mosh가 설치되어 있어야 합니다. 개발팀에 확인하세요.

입력 모드 활용하기

여러 줄 입력하기

긴 질문이나 여러 내용을 한 번에 전달하고 싶을 때:

💬 AI한테 이렇게 말하세요:

첫 줄 입력 → **Shift + Enter** 로 줄바꿈 → 다음 줄 입력 → 완성되면 **Enter**

파일 내용을 AI한테 바로 보내기

에러 로그나 설정 파일 내용을 AI한테 보내고 싶을 때:

파일 내용을 복사 (**Ctrl+C** 또는 **Cmd+C**)

Claude Code에 붙여넣기 (**Ctrl+V** 또는 **Cmd+V**)

"이 내용에서 문제를 찾아줘"라고 말하기

스크린샷을 AI한테 보내기

화면에 보이는 에러를 AI한테 보여주고 싶을 때:

스크린샷 파일 경로를 알려주기

💬 "이 스크린샷 봐줘 /tmp/screenshot.png"

자주 쓰는 조합

작업	키 조합
질문 취소하고 다시 쓰기	Escape → 새로 입력
AI 멈추고 다시 질문	Ctrl+C → 새로 입력
파일 경로 빠르게 입력	경로 앞부분 입력 → Tab
이전 질문 재사용	↑ → 수정 → Enter
완전히 종료	Ctrl+D 또는 /exit

문제 해결

답변이 너무 길어서 끝나지 않을 때

`Ctrl + C` 로 답변을 중단한 뒤:

같은 주제를 이어가려면: "짧게 요약해서 답해줘"

새 주제로 넘어가려면: `/clear` 로 대화 초기화 (00장 참고)

터미널 화면이 깨졌을 때 (글자가 이상하게 보임)

터미널에 `reset` 을 입력하고 엔터를 치세요. 화면이 정리됩니다.

💡 `reset` 이 안 되면 터미널을 닫았다 다시 열면 됩니다.

이전 대화를 이어가고 싶을 때

Claude Code를 끝냈다가 다시 같은 대화를 이어가고 싶을 때:

```
claude --resume latest
```

alias를 설정했다면 `ccr` 로 더 짧게 쓸 수 있습니다.

Tab으로 자동완성이 안 될 때

파일 경로를 수동으로 직접 입력하세요:

```
"src/components/Button.tsx 확인해줘"
```

또는 AI한테 파일 경로를 물어보세요:

```
"Button 컴포넌트 파일 경로가 뭐야?"
```

명령어를 잘못 입력했을 때

`Escape` 를 누르면 현재 입력한 내용이 취소됩니다.

💡 정리

- ✓ `Escape` , `Ctrl + C` , `Tab` — 이 3개만 기억하면 충분
 - ✓ `alias`로 명령어 줄이기 — AI한테 설정 시키면 됨
 - ✓ 여러 창 필요하면 — AI한테 `tmux` 세팅 시키기
 - ✓ 원격 접속이 불안정하면 — AI한테 `mosh` 설정 시키기
 - ✓ 이전 대화 이어가기 — `claude --resume latest` (또는 `ccr`)
-
-

다음 단계

03장 — 내 취향대로 설정하기 — AI 동작 방식 바꾸기

내 취향대로 설정하기 — AI 환경 설정

설정 파일은 **AI 비서의 업무 매뉴얼** 같은 것입니다. 새 직원한테 "이건 이렇게 해"라고 알려주듯, AI한테 규칙을 정해줄 수 있습니다.

모든 설정은 AI한테 자연어로 시키면 됩니다. 설정 파일을 직접 편집할 필요 없습니다.

매번 허락 받는 게 귀찮을 때

이런 상황일 때

AI가 파일 수정할 때마다 "이거 할까요?"라고 물어봐서 방해가 될 때.

3가지 모드

모드	설명	추천 상황	비유
default	파일 수정할 때마다 승인 요청	처음 배울 때, 중요한 작업	"일단 보고하고 하겠습니다"
plan	읽기만 하고 수정 안 함 (분석 전용)	코드 분석, 구조 파악만 할 때	"보기만 합니다"
bypassPermissions	AI가 자동으로 파일 수정 (문지 않음)	신뢰할 수 있는 작업, 빠르게 진행	"알아서 하겠습니다"

💬 AI한테 이렇게 말하세요:

"모드를 bypassPermissions로 바꿔줘"

"분석만 할 거니까 plan 모드로 바꿔줘"

"다시 default 모드로 바꿔줘"

어떤 모드를 써야 할까요?

상황	추천 모드
Claude Code를 처음 쓸 때	default
코드를 분석만 하고 싶을 때	plan
익숙해져서 빠르게 작업하고 싶을 때	bypassPermissions
중요한 프로덕션 코드를 수정할 때	default
간단한 반복 작업을 맡길 때	bypassPermissions

⚠️ `bypassPermissions`는 AI가 마음대로 파일을 수정합니다. Git을 쓰면 언제든지 되돌릴 수 있으니 안전합니다. Git 사용법은 05장을 참고하세요.

💡 처음 쓰는 분은 **default** 모드로 시작하세요. AI가 뭘 하려는지 하나하나 확인하면서 익힐 수 있습니다. 익숙해지면 `bypassPermissions`로 바꿔서 시간을 절약하세요.

AI가 자꾸 느려질 때

이런 상황일 때

AI가 예전 대화를 다 기억하고 있어서 응답이 느려질 때, AI는 모든 대화를 "기억력"에 쌓고 있습니다. 기억력이 가득 차면 느려집니다.

자동 기억력 정리 설정

💬 AI한테 이렇게 말하세요:

"자동 메모리 정리 설정해줘. 기억력 75%에서 자동 정리되게"

AI가 설정 파일을 알아서 수정합니다. 이후 기억력이 75%를 넘으면 자동으로 정리됩니다.

수동 정리

수동으로 정리하고 싶다면:

`/compact` — 대화를 요약해서 기억력 정리 (진행상황 유지)

`/clear` — 대화 전부 삭제 (완전 초기화)

상세 설명은 00장을 참고하세요.

기억력 관리 팁

습관	효과
한 기능 완성 → /clear	기억력 100% 복구
같은 주제 계속 → 중간에 /compact	기억력 50% 확보
기억력 75% 넘으면 → /compact	속도 저하 방지

자세한 내용은 10장을 참고하세요.

작업 현황을 한눈에 보고 싶을 때

이런 상황일 때

지금 AI가 뭘 하는지, 기억력은 얼마나 쓰고 있는지, 비용은 얼마나 드는지 궁금할 때.

💬 AI한테 이렇게 말하세요:

"상태 표시줄에 기억력 퍼센트랑 비용 보여주게 설정해줘"

설정하면 화면 아래에 항상 현재 상태가 표시됩니다.

/status로 즉시 확인

/status 를 입력하면 현재 상태를 한눈에 볼 수 있습니다:

프로젝트 경로
현재 모드 (default / plan / bypassPermissions)
기억력 사용량
이번 세션 비용

파일 수정 후 자동으로 뭔가 하고 싶을 때

이런 상황일 때

파일을 저장할 때마다 자동으로 코드 정리를 하거나, 커밋(저장)할 때마다 테스트를 돌리고 싶을 때.

이런 자동 실행 규칙을 "훅(Hook)"이라고 합니다. 무언가 발생하면 자동으로 다른 작업이 실행되는 것입니다.

💬 AI한테 이렇게 말하세요:

"파일 저장할 때마다 자동으로 코드 정리되게 훅 설정해줘"

"커밋할 때마다 자동으로 테스트 실행되게 설정해줘"

"PR 만들 때마다 자동으로 코드 리뷰 돌리게 설정해줘"

AI가 설정 파일에 훅을 추가합니다.

유용한 훅 예시

이벤트	자동 실행	효과
파일 저장	코드 정리 (lint)	코드 스타일 통일
커밋 전	테스트 실행	버그 방지
PR 생성	코드 리뷰	품질 확인

💡 처음에는 훅 없이 단순하게 시작하세요. "매번 이 작업 반복하는데..." 싶을 때 그때 설정하면 됩니다.

설정 우선순위

규칙이 겹치면 어느 것을 따를까요?

- 1순위: 회사 규칙 (모두 따라야 함)
- 2순위: 프로젝트 규칙 (이 프로젝트만 따름)
- 3순위: 내 개인 규칙 (나만 적용)

예를 들어:

회사 규칙: "파일 수정할 때마다 승인 받아야 함 (default 모드)"

프로젝트 규칙: "이 프로젝트는 빠르게 (bypassPermissions 모드)" → 프로젝트 규칙이 적용되어 이 프로젝트에서는 자동 승인됩니다.

회사 규칙이 있으면 반드시 따르세요. 프로젝트별로 다르게 하고 싶으면 팀장에게 먼저 상의하세요.

자주 묻는 질문

Q: 설정을 바꾸면 프로젝트가 망가지나요? A: 아니요. 설정은 AI의 행동만 바꿀 뿐, 코드 자체는 건드리지 않습니다. 실수해도 Git으로 언제든지 되돌릴 수 있습니다.

Q: 회사와 개인 설정이 충돌하면? A: 회사 규칙이 우선합니다. 프로젝트마다 다르게 하고 싶으면 팀장과 상의하세요.

Q: 메모리 정리하면 지난 대화를 못 보나요? A: `/compact` 는 중요한 내용을 요약해서 남깁니다. 세부 내용만 정리됩니다. `/clear` 는 전부 삭제됩니다.

Q: 설정을 원래대로 되돌리고 싶어요. A: AI한테 "설정을 기본값으로 되돌려줘"라고 하세요.

실전 예시

상황: "매일 아침 AI가 느려서 답답함"

해결 전:

Claude Code 실행 → 대화 시작 → 점점 느려짐 → 답답

해결 후:

Claude Code 실행 → 대화 시작

한 기능 완성 → `/compact` 로 정리

빠른 속도 유지!

상황: "매번 승인 묻는 게 방해됨"

💬 AI한테 이렇게 말하세요:

"모드를 `bypassPermissions`로 바꿔줘"

→ AI가 자동으로 수정하고 결과만 보여줌 (승인 요청 없음)

상황: "코드 분석만 하고 수정은 안 하고 싶어"

💬 AI한테 이렇게 말하세요:

"plan 모드로 바꿔줘. 분석만 할 거야"

→ AI가 코드를 읽고 분석만 하고, 수정하지 않음

💡 실천 팁

- ✅ 처음엔 default 모드로 시작
 - ✅ 익숙해지면 bypassPermissions로 시간 절약
 - ✅ 느려지면 `/compact` (00장 참고)
 - ✅ 설정 변경은 AI한테 자연어로 시키기
 - ✅ 백업은 Git으로 자동 관리
 - ❌ 중요한 파일은 Git 백업 없이 수정하지 말 것
 - ❌ 설정 파일을 직접 편집하지 말 것 — AI한테 시키세요
-

다음 단계

04장 — AI에게 전문 기술 가르치기에서 스킬과 커맨드를 배울 수 있습니다.

AI에게 전문 기술 가르치기 — 스킬과 커맨드

스킬이 뭔가요?

스킬은 **AI에게** 특정 업무 매뉴얼을 주는 것입니다.

신입사원에게 "이건 이 매뉴얼 보고 해"라고 건네주듯, `/design-review` 라고 치면 AI가 "디자인 리뷰 전문가" 모드로 변신합니다.

스킬 vs 그냥 시키기 — 뭐가 다른가요?

구분	그냥 시키기	스킬 사용
품질	보통	높음 (매뉴얼 기반)
일관성	매번 달라질 수 있음	항상 같은 기준
예시	"디자인 봐줘"	<code>/design-review</code> [스크린샷]"

우리 회사에서 쓸 수 있는 스킬

새 페이지 디자인이 필요할 때 → `/design-plan`

💬 AI한테 이렇게 말하세요:

`/design-plan` 결제 화면 재설계해줘. 현재 3단계인데 1단계로 줄이고 구글페이 추가하고 싶어. 사용자가 중간에 포기하는 게 문제야"

AI가 해주는 것:

인터뷰: "사용자가 제일 먼저 봐야 할 것은?"

디자인 생성: 레이아웃 스케치

검토: 버튼 위치, 폰트 크기 등 개선 제안

문서화: 최종 설계를 DESIGN.md로 저장

✅ **잘 쓰는 법:** 현재 상태, 목표, 이유를 함께 알려주세요.

현재: "지금 결제가 3단계"

목표: "1단계로 줄이고 싶어"

이유: "사용자가 중간에 포기해서"

❌ **안 좋은 예:** "UI 디자인 해줘" (너무 막연함 → 결과 부실)

기존 화면 디자인을 검토하고 싶을 때 → /design-review

💬 AI한테 이렇게 말하세요:

"/design-review [스크린샷 첨부] 이 로그인 폼 사용하기 편해 보여? 빨간 경고가 눈에 띄어?"

AI가 스크린샷을 분석하고:

"버튼이 너무 작다 — 터치 영역 최소 44px 권장"

"색상 대비가 약하다 — 접근성 기준 미달"

"로딩 상태 표시가 없다 — 사용자가 응답 여부를 모름"

코드 레벨 수정안까지 제시합니다.

💡 꼭 스크린샷과 함께 사용하세요. 텍스트만으로는 디자인을 검토할 수 없습니다. 💡 스크린샷 찍기는 06장 — 브라우저 자동화를 활용하면 더 편합니다.

코드를 리뷰하고 싶을 때 → /review-ai

💬 AI한테 이렇게 말하세요:

"/review-ai src/payment/stripe.ts 보안이랑 성능 중심으로 검토해줘"

"/review-ai src/auth/ 폴더 전체 보안 점검해줘"

AI가 해주는 것:

버그 찾기: "null check 빠짐 — 42번 줄"

보안 검토: "비밀번호가 평문으로 저장됨 — 해싱 필요"

성능 분석: "N+1 쿼리 문제 — DB 호출 100번 → 1번으로 줄일 수 있음"

개선 코드 제시: 수정된 코드와 설명

✅ **잘 쓰는 법:** 파일 경로를 정확히 + 중점적으로 볼 부분 지정

❌ **안 좋은 예:** "코드 검토해줘" (뭘 검토할지 몰라서 결과가 산만함)

자동 배포를 설정하고 싶을 때 → /cicd-setup

💬 AI한테 이렇게 말하세요:

"/cicd-setup React + Node.js 프로젝트. GitHub 푸시하면 자동 테스트하고 통과하면 자동 배포해 줘. Slack 알림도 보내줘"

AI가 GitHub Actions 워크플로우, 테스트 설정, 배포 스크립트, 문서까지 한 번에 생성합니다.

자세한 내용은 11장 — CI/CD를 참고하세요.

커맨드 — 큰 작업을 한 번에

큰 작업을 자율적으로 맡기고 싶을 때 → /ultrawork

💬 AI한테 이렇게 말하세요:

"/ultrawork TypeScript 타입 에러 전부 0개로 만들어줘. src/ 폴더 전체"

"/ultrawork 사용자 인증 시스템 전체 리팩토링. OAuth 추가하고 보안 감사 포함해줘"

AI가 3단계를 자동으로 진행합니다:

계획(Planning) — 작업 분석, 리스크 식별, 테스트 계획

실행(Execution) — 코드 작성, 테스트 실행, 오류 수정

검증(Verification) — 최종 테스트, 성능 측정, 문서 작성

결과물: 완성된 코드 + 테스트 결과 + 사용 가이드까지 한 번에 받을 수 있습니다.

/ultrawork는 언제 쓰나요?

상황	/ULTRAWORK 사용	직접 시키기
파일 1개 수정	❌ 과하다	✅
폴더 전체 리팩토링	✅	❌ 너무 복잡
새 기능 전체 개발	✅	❌ 너무 오래 걸림
간단한 버그 수정	❌ 과하다	✅

여러 팀이 동시에 작업해야 할 때 → /setup-workspace

💬 AI한테 이렇게 말하세요:

"/setup-workspace 프론트엔드, 백엔드, QA 동시 작업 세팅해줘"

AI가 터미널을 여러 칸으로 분할하고 각각에 에이전트를 배치해서 동시에 진행합니다.

자세한 내용은 08장 — 멀티에이전트를 참고하세요.

나만의 스킬 만들기

이런 상황일 때

"우리 회사는 항상 커밋 메시지를 이렇게 쓴다", "새 페이지 만들 때는 항상 이 템플릿을 쓴다" — 이런 반복 작업이 많을 때.

💬 AI한테 이렇게 말하세요:

"커밋 메시지 스킬 만들어줘. 제목은 50자 이내, 본문은 72자씩 줄바꿈, 이슈 번호 포함하는 규칙이야"

"새 React 컴포넌트 만드는 스킬 만들어줘. 항상 테스트 파일도 함께 생성하고, Storybook도 추가하는 규칙으로"

AI가 `.claude/skills/` 폴더에 스킬 파일을 자동으로 생성합니다. 이후 `/commit-message` 처럼 호출할 수 있습니다.

💡 자주 반복하는 "매뉴얼 같은 작업"을 스킬로 만들면, 팀 전체가 같은 규칙을 따르게 됩니다. 혼동이 없어집니다.

설치된 스킬 확인하기

💬 AI한테 이렇게 말하세요:

"설치된 스킬 보여줘"

AI가 현재 프로젝트에서 사용 가능한 모든 스킬 목록을 보여줍니다.

자주 묻는 질문

- Q: 스킬을 잘못 만들었어. 수정할 수 있나요?** A: AI한테 "이 스킬 수정해줘"라고 하면 기존 스킬 파일을 업데이트합니다.
- Q: 다른 프로젝트에서도 같은 스킬을 쓰고 싶어요.** A: AI한테 "이 스킬을 글로벌로 설정해줘"라고 하면 모든 프로젝트에서 사용 가능합니다.
- Q: 스킬 없이 그냥 시키면 안 되나요?** A: 됩니다. 하지만 스킬을 쓰면 AI가 전문가 모드로 더 정확하게 처리합니다. 반복 작업일수록 스킬이 효과적입니다.
- Q: 팀원도 내가 만든 스킬을 쓸 수 있나요?** A: 네. 스킬 파일이 Git으로 공유되면 팀원도 같은 스킬을 사용할 수 있습니다.

스킬 요약표

스킬	용도	핵심
/design-plan	새 디자인 설계	현재상태 + 목표 + 이유
/design-review	디자인 검토	스크린샷 필수
/review-ai	코드 리뷰	파일경로 + 중점사항
/cicd-setup	자동 배포 설정	기술스택 + 배포대상
/ultrawork	큰 작업 자동 처리	구체적 목표 설명
/setup-workspace	동시 작업 세팅	팀 구성 설명

💡 실천 팁

- ✅ 스킬 호출 시 상황과 목표를 구체적으로
- ✅ `/design-review` 는 꼭 스크린샷과 함께
- ✅ `/review-ai` 는 파일 경로를 정확하게
- ✅ 자주 쓰는 작업은 커스텀 스킬로 만들기
- ✅ 큰 작업은 `/ultrawork` 로 한 번에
- ❌ 스킬 이름만 치고 설명 없이 사용하지 말 것

다음 단계

05장 — 코드 변경사항 관리하기에서 Git과 PR을 배울 수 있습니다.

코드 변경사항 관리하기 — Git 기초

Git이 뭔가요?

Git(깃)은 코드의 버전 관리 도구입니다. 구글독스의 "버전 기록"과 비슷합니다.

구분	구글 독스	GIT
뭘 관리?	문서	코드 파일들
되돌리기	"버전 기록" 클릭	AI한테 "되돌려줘"
여러 명 작업	동시 편집	각자 수정 후 합치기
변경 이력	"누가 수정했는지" 표시	커밋 기록으로 추적

왜 필요한가요?

누가 언제 뭘 바꿨는지 추적 — 나중에 "누가 이걸 바꿨어?" 궁금할 때 Git 기록에서 찾을 수 있음

실수해도 되돌리기 가능 — 예전 버전으로 언제든지 복구

여러 사람이 동시에 작업 가능 — 각자 다른 파일을 수정하고 Git이 합쳐줌

모든 Git 작업은 AI한테 자연어로 시키면 됩니다. 명령어를 외울 필요 없습니다.

Git 핵심 용어 3개

용어	설명	비유
커밋(commit)	변경사항을 저장하는 것	구글독스의 "버전 기록 만들기"
PR(Pull Request)	"이 코드 검토해주세요"라는 팀 공유 요청	"보고서 검토 부탁드립니다"
브랜치(branch)	코드를 따로 분리해서 작업하는 공간	별도 복사본에서 작업

AI한테 Git 시키기

지금까지 바꾼 거 저장하고 싶을 때

💬 AI한테 이렇게 말하세요:

"커밋해줘"

AI가 자동으로:

- 뭘 바꿨는지 확인
- 커밋 메시지 작성 (변경 내용을 요약한 제목)
- 저장 완료

더 구체적으로 말할 수도 있습니다:

💬 AI한테 이렇게 말하세요:

"커밋해줘. 로그인 폼 추가하고 인증 로직 구현한 거야"

💡 AI가 자동으로 적절한 커밋 메시지를 만들어주지만, 설명을 더하면 더 명확한 메시지를 만듭니다.

얼마나 자주 커밋해야 하나요?

타이밍	추천
한 기능 완성했을 때	✅ 커밋하세요
세션 전환 전	✅ 꼭 커밋하세요
퇴근 전	✅ 꼭 커밋하세요
코드 한 줄 수정할 때마다	❌ 너무 자주

바꾼 내용을 보고 싶을 때

💬 AI한테 이렇게 말하세요:

"지금까지 뭐 바뀌었는지 보여줘"

AI가 보여주는 것:

- 수정된 파일 목록
- 각 파일에서 몇 줄이 추가/삭제됐는지
- 새로 만들어진 파일, 삭제된 파일

💬 더 자세히 보고 싶으면:

"지난 5개 커밋 보여줘"

"오늘 바뀐 파일 전부 보여줘"

이전 상태로 돌리고 싶을 때

💬 AI한테 이렇게 말하세요:

"아까 커밋이 잘못됐어. 되돌려줘"

AI가 새로운 수정 커밋을 만들어서 안전하게 되돌립니다. 기존 기록은 그대로 유지됩니다.

💡 AI도 위험한 되돌리기 요청은 먼저 확인을 물어봅니다. "이전 3개 커밋 전부 되돌릴까요?"처럼요.

특정 시점으로 돌아가고 싶을 때

💬 AI한테 이렇게 말하세요:

"3일 전 상태로 돌려줘"

"로그인 기능 추가하기 전으로 돌려줘"

AI가 커밋 기록에서 해당 시점을 찾아서 안전하게 되돌립니다.

변경사항을 팀에 공유하고 싶을 때

💬 AI한테 이렇게 말하세요:

"main으로 PR 만들어줘. 제목은 '로그인 기능 추가'"

PR(Pull Request)이란 "이 코드 검토해주세요"라는 팀 공유 요청입니다.

AI가 해주는 것:

변경사항 정리

PR 제목과 설명 작성

GitHub에 업로드

팀이 검토 → 승인 → 코드가 합쳐짐

PR은 왜 필요한가요?

다른 사람이 내 코드를 검토해서 버그를 미리 잡을 수 있음

코드 변경 이력이 깔끔하게 남음

팀 전체가 어떤 변경이 있었는지 알 수 있음

코드 리뷰를 받고 싶을 때

💬 AI한테 이렇게 말하세요:

```
"/review-ai src/payment.ts 보안이랑 성능 중심으로"
```

자세한 내용은 04장 — 스킬과 커맨드를 참고하세요.

커밋 메시지 규칙

커밋 메시지는 "일기장 제목"처럼 명확해야 나중에 찾기 쉽습니다. AI한테 커밋을 시키면 자동으로 이 규칙을 따릅니다.

타입	의미	예시
feat	새 기능 추가	feat(auth): 구글 로그인 추가
fix	버그 수정	fix(payment): 결제 오류 해결
refactor	코드 정리 (기능 변화 없음)	refactor(api): 에러 처리 개선
docs	문서 추가/수정	docs: README 업데이트
test	테스트 추가	test: 로그인 flow 테스트
chore	빌드, 설정 변경	chore(deps): 라이브러리 업그레이드

💡 AI한테 "커밋해줘"라고만 하면 변경 내용을 분석해서 자동으로 이 형식에 맞춰 메시지를 작성합니다. 외울 필요 없습니다.

실전 흐름: 기능 구현 → 저장 → 공유

💬 AI한테 이렇게 말하세요:

1단계 — 기능 구현:

"사용자 검색 기능 만들어줘"

2단계 — 커밋:

"커밋해줘"

3단계 — 팀에 공유:

"main으로 PR 만들어줘"

4단계 — 피드백 반영 (동료가 "페이지네이션 추가해"라고 했을 때):

"검색 결과에 페이지네이션 추가해줘"

5단계 — 다시 커밋:

"수정한 거 커밋하고 PR 업데이트해줘"

⚠️ 절대 하면 안 되는 것

금지 행위	왜 위험한가	안전한 방법
"강제로 푸시해줘" (force push)	팀의 작업이 다 날아감	"새 커밋으로 수정해줘"
다른 사람이 수정 중인 파일 건드리기	코드 충돌 발생	팀과 먼저 상의
중요 파일 무분별하게 수정	서비스 장애 가능	PR로 검토 후 승인 받기

💡 AI도 파괴적인 요청은 먼저 확인을 물어봅니다. "다 지우고 새로 해줘" 같은 요청에도 "영향: XX. 진행할까요?"라고 안전장치가 작동합니다.

자주 묻는 질문

Q: 커밋을 잘못했어. 어떻게 하지? A: AI한테 "아까 커밋이 잘못됐어. 수정해줘"라고 하세요. 새로운 수정 커밋을 만들어서 안전하게 고칩니다.

Q: PR이 너무 많은 파일을 건드려. 괜찮아? A: PR은 작을수록 좋습니다. "이 기능을 2개 PR로 나눠줘"라고 시키세요.

Q: 팀이 PR을 거절했어. 어떻게 해? A: 피드백을 읽고 AI한테 수정을 시키세요. "함수가 너무 길대. 더 작게 나눠줘"

Q: 커밋과 푸시의 차이가 뭐야? A: 커밋은 내 컴퓨터에 저장하는 것, 푸시는 GitHub에 올리는 것입니다. AI한테 "커밋하고 푸시해줘"라고 하면 둘 다 합니다.

실천 팁

- ✅ 커밋은 자주, 작게 — "한 기능 = 한 커밋"
 - ✅ PR도 자주, 작게 — "한 기능 = 한 PR"
 - ✅ 세션 전환 전, 퇴근 전에 반드시 커밋
 - ✅ 모든 Git 작업은 AI한테 자연어로
 - ❌ force push는 절대 금지
 - ❌ 남의 작업 무시하고 덮어쓰지 말 것
-

다음 단계

06장 — 브라우저 자동화에서 AI가 웹사이트를 직접 조작하는 법을 배울 수 있습니다.

AI가 웹사이트를 직접 조작하게 하기 — 브라우저 자동화

이게 뭔가요?

브라우저 자동화는 **AI가 크롬 브라우저를 열고, 클릭하고, 스크린샷 찍는 것**입니다.

일반 AI에게 "우리 사이트 로그인 되나?"라고 물으면 "됩니다"라고 대충 답합니다 (실제로 안 봤는데). Claude Code는 실제로 브라우저를 열어서 로그인을 시도하고, 스크린샷을 찍어서 확실하게 확인합니다.

비유

구분	일반 AI	CLAUDE CODE (브라우저 자동화)
"사이트 정상이야?"	"아마 정상일 겁니다"	[스크린샷] "확인해봤는데 정상입니다"
"로그인 돼?"	"코드상 될 것 같습니다"	"직접 로그인해봤는데 됩니다"
"버튼 위치 괜찮아?"	"보통 오른쪽 하단이 좋습니다"	[스크린샷] "현재 위치가 최적입니다"

언제 쓰나요?

우리 사이트 스크린샷이 필요할 때

💬 AI한테 이렇게 말하세요:

"https://myapp.com 로그인 화면 스크린샷 찍어줘"

"우리 사이트 모바일 화면(375px) 스크린샷도 찍어줘"

"다크 모드에서 어떻게 보이는지 스크린샷 찍어줘"

AI가 해주는 것:

브라우저 자동 열기

사이트 접속

스크린샷 촬영

파일로 저장

사이트가 정상 작동하는지 확인하고 싶을 때

💬 AI한테 이렇게 말하세요:

"우리 사이트 로그인 되는지 확인해봐. 테스트 계정으로"

"회원가입 → 로그인 → 프로필 수정 순서로 전부 되는지 확인해줘"

AI가 실제로 버튼을 클릭하고, 입력창에 글자를 넣고, 결과를 확인합니다.

확인 결과 보고 예시

AI가 이렇게 보고합니다:

"로그인 페이지:  정상 (1.2초)"

"회원가입 페이지:  비밀번호 확인 입력 후 에러 발생"

[에러 스크린샷 첨부]

외부 서비스에서 설정을 바꿔야 할 때

💬 AI한테 이렇게 말하세요:

"Google Cloud 콘솔에서 OAuth URI에 `http://localhost:3000/callback` 추가해줘"

AI가 해주는 것:

Google 콘솔 접속 (자동 로그인)

설정 페이지 이동

URI 입력란 찾기

새 URI 입력

저장 버튼 클릭

로그인이 필요한 사이트 — 처음 한 번만 로그인하면 이후 자동

이런 상황일 때

Google Cloud 콘솔이나 GitHub 같은 곳에서 매번 로그인하기 귀찮을 때.

💬 AI한테 이렇게 말하세요:

"Google Cloud 콘솔에 로그인해줘"

AI가 크롬을 열고 로그인을 진행합니다. 로그인 정보가 Chrome 프로필에 저장되어 다음부터는 자동으로 로그인됩니다.

💡 로그인 정보는 Mac의 Keychain(암호 저장소)에 안전하게 저장됩니다.

2단계 인증(2FA)이 필요한 경우

일부 사이트는 로그인 후 휴대폰 인증이 필요합니다. 이 경우:

AI가 로그인 시도

"인증 코드를 입력해주세요"라고 안내

당신이 인증 코드를 입력

이후부터는 자동

AI용 브라우저는 따로 있어요

이런 상황일 때

AI가 웹사이트를 자동으로 조작하는 동안, 내가 일반 크롬을 쓰고 싶을 때.

걱정하지 마세요. AI는 **Chromium**(AI 전용 가벼운 브라우저)을 사용하고, 당신은 일반 Chrome을 사용합니다.

구분	AI의 브라우저	당신의 브라우저
종류	Chromium (AI 전용)	Chrome (개인용)
쿠키/로그인	별도 저장	별도 저장

구분	AI의 브라우저	당신의 브라우저
충돌	없음	없음

따로 설정할 필요도 없습니다. 자동으로 분리됩니다.

자주 묻는 질문

Q: 브라우저 자동화를 별도로 설치해야 하나요? A: Claude Code를 설치하면 기본 포함됩니다. 안 되면 AI한테 "Playwright 설치해줘"라고 하세요.

Q: 어떤 사이트든 자동화할 수 있나요? A: 대부분 가능합니다. 하지만 CAPTCHA(로봇 방지 인증)가 있는 사이트는 자동화가 어려울 수 있습니다.

Q: AI가 내 개인 크롬 북마크나 비밀번호를 볼 수 있나요? A: 아니요. AI는 별도의 Chromium 브라우저를 사용하므로 당신의 Chrome과 완전히 분리되어 있습니다.

Q: 모바일 화면도 테스트할 수 있나요? A: 네. "모바일 화면(375px)으로 스크린샷 찍어줘"라고 하면 모바일 크기로 확인합니다.

💡 실제 활용 예시

예시 1: 웹사이트 모니터링

💬 AI한테 이렇게 말하세요:

"우리 사이트 홈페이지, 로그인, 결제 페이지 다 정상인지 확인해줘"

AI가 각 페이지를 자동으로 방문하고 결과를 보고합니다.

예시 2: 배포 후 확인

💬 AI한테 이렇게 말하세요:

"배포 끝났으니까 사이트 들어가서 새 버튼이 잘 보이는지 스크린샷 찍어줘"

AI가 배포된 사이트를 방문하고 스크린샷으로 확인합니다.

예시 3: 디자인 리뷰와 함께 사용

💬 AI한테 이렇게 말하세요:

"우리 사이트 스크린샷 찍고, 그거 가지고 /design-review 해줘"

스크린샷 캡처 → 디자인 검토까지 한 번에 가능합니다. (04장 참고)

예시 4: 경쟁사 모니터링

💬 AI한테 이렇게 말하세요:

"경쟁사 사이트 스크린샷 찍고 어제 찍은 것과 뭐가 바뀌었는지 비교해줘"

예시 5: 폼 입력 자동화

💬 AI한테 이렇게 말하세요:

"테스트 데이터로 회원가입 폼을 10번 제출해줘. 각각 다른 이메일로"

⚠️ 주의사항

로그인 정보 관리

- ✅ Mac Keychain에 저장
- ✅ 테스트 계정 별도 생성 (예: test@내계정.kr)
- ✅ 강한 비밀번호 사용
- ❌ 실제 사용자 계정으로 테스트하지 말 것
- ❌ 신용카드 정보 입력하지 말 것
- ❌ 실제 사용자의 이메일/비밀번호 입력 금지

사이트 부하 주의

너무 자주 자동 접속하면 봇으로 탐지될 수 있습니다.

필요한 만큼만 실행

시간 간격 두기

과도한 자동화 피하기

💡 실천 팁

- ✓ 스크린샷은 `/design-review` 와 함께 활용
 - ✓ 로그인은 테스트 계정으로
 - ✓ 자동화는 반복되는 작업부터 시작
 - ✓ 문제 발생 시 스크린샷으로 기록
 - ✓ 배포 후에는 반드시 브라우저로 확인
 - ✗ 실제 사용자 계정으로 테스트 금지
 - ✗ 과도한 자동화로 사이트에 부하 주지 말 것
-
-

다음 단계

07장 — 외부 도구 연결에서 GitHub, 데이터베이스, 웹 검색 등을 AI가 직접 사용하는 법을 배울 수 있습니다.

AI에게 회사 시스템 접근 권한 주기 — 외부 도구 연결

MCP가 뭔가요?

MCP(Model Context Protocol)는 **AI가 외부 서비스에 접근하는 통로**입니다. USB 포트처럼 생각하면 됩니다.

컴퓨터에 프린터를 연결하면 인쇄할 수 있듯, AI에 GitHub을 연결하면 PR을 만들 수 있습니다.

작업	MCP 없으면	MCP 있으면
GitHub PR 만들기	"이런 명령어 쓰세요" (설명만)	"PR 생성 완료!" (직접 실행)
데이터베이스 조회	"직접 SQL 쓰세요"	"지난달 매출: 5,000만원"
웹 검색	링크만 제시	최신 검색 결과 표시
라이브러리 문서	예전 기억 사용 (부정확할 수 있음)	최신 공식 문서 로드
Slack 메시지	"직접 보내세요"	"팀 채널에 메시지 보냈습니다"

개발자가 처음 한 번 연결해두면, 이후로는 자연어로 요청만 하면 됩니다.

비유로 이해하기

MCP 없는 AI = 전화만 되는 스마트폰 MCP 있는 AI = 앱을 설치한 스마트폰 (카카오톡, 은행앱, 지도앱...)

앱(MCP)을 설치할수록 AI가 할 수 있는 일이 늘어납니다.

연결하면 뭘 할 수 있나요?

GitHub에서 코드 관리하고 싶을 때

💬 AI한테 이렇게 말하세요:

"GitHub PR 만들어줘. 제목: 로그인 기능 추가"

"우리 저장소의 버그 레이블 이슈 모두 보여줘"

"이 PR의 코드 충돌 자동으로 해결해줘"

"이 이슈에 '작업 시작합니다' 댓글 달아줘"

AI가 GitHub에 직접 접속해서 PR 생성, 이슈 조회, 충돌 해결, 댓글 작성까지 자동으로 처리합니다.

💡 Git 관련 기본 사용법은 05장을 참고하세요.

데이터베이스에서 통계를 뽑아야 할 때

💬 AI한테 이렇게 말하세요:

"지난달 매출 top 5 상품 보여줘"

"4월 신규 가입자는 몇 명이야?"

"최근 7일간 일일 활성 사용자 추이 보여줘"

"이번 달과 지난달 매출 비교해줘"

AI가 SQL 쿼리를 작성하고 실행해서 결과를 보기 좋게 정리합니다.

⚠️ AI는 데이터 조회(**SELECT**)만 합니다. 데이터를 삭제하거나 구조를 바꾸지 않으니 안심하세요.

최신 정보를 찾아야 할 때

💬 AI한테 이렇게 말하세요:

"React 19의 새로운 기능이 뭐야?"

"AI 관련 최신 보안 위협 찾아줘"

"이 에러 메시지를 웹에서 검색해줘: ECONNREFUSED"

"Next.js 15 마이그레이션 가이드 찾아줘"

AI가 웹을 검색하고 신뢰할 수 있는 출처의 최신 정보를 정리합니다.

💡 AI의 기본 지식은 학습 시점까지의 정보만 알고 있습니다. 최신 정보가 필요하면 반드시 웹 검색을 활용하세요.

회사 위키 문서를 찾아야 할 때

💬 AI한테 이렇게 말하세요:

"온보딩 문서에서 개발 환경 설정 부분 찾아줘"

"우리 API 가이드 문서를 읽고 인증 부분 설명해줘"

"지난 회의록에서 결제 관련 결정사항 찾아줘"

AI가 Confluence 등 위키를 검색해서 필요한 내용을 요약합니다.

라이브러리 공식 문서가 필요할 때

💬 AI한테 이렇게 말하세요:

"Prisma에서 관계형 쿼리 어떻게 쓰는지 공식 문서 확인해줘"

"Tailwind CSS v4 변경사항 알려줘"

AI가 공식 문서에서 최신 정보를 가져와서 코드 예제와 함께 설명합니다.

설정은 누가 하나요?

개발자가 처음 한 번만 연결 설정을 합니다. 이후에는 자연어로 요청만 하면 됩니다.

현재 연결된 서비스

각 프로젝트마다 다를 수 있습니다. 현재 상태를 확인하려면:

💬 AI한테 이렇게 말하세요:

"지금 연결된 MCP 서비스 보여줘"

일반적으로 연결 가능한 서비스:

서비스	용도	상태
GitHub	PR, Issue 자동 관리	✅ 연결됨
웹 검색 (Brave)	최신 정보 검색	✅ 연결됨
Playwright	브라우저 자동화	✅ 연결됨 (06장 참고)
데이터베이스	통계 조회	프로젝트마다 다름
Confluence	위키 문서 검색	프로젝트마다 다름
Slack	메시지 전송	프로젝트마다 다름

더 연결하고 싶으면?

개발팀에 말하세요:

"DBHub으로 데이터베이스 접근 권한 줄 수 있나요?"

"Confluence 문서를 AI가 읽을 수 있게 해줄 수 있나요?"

"Slack MCP를 연결해서 AI가 알림을 보낼 수 있게 해줄 수 있나요?"

💡 새 서비스 연결은 개발팀이 10~30분이면 처리합니다. 부담 없이 요청하세요.

⚠️ 보안 주의사항

데이터베이스 접근 제한

- ✅ 데이터 조회(SELECT)만 가능하게 설정
- ✅ 매출, 사용자 수 등 통계 조회 OK
- ❌ 데이터 삭제(DELETE), 구조 변경(DROP)은 금지
- ❌ 사용자 비밀번호, 신용카드 번호, 개인 주소 절대 조회 금지

→ 개발팀이 권한을 "읽기만 가능"으로 제한해야 합니다.

토큰(접속 열쇠) 관리

- ✅ 매 분기마다 토큰 재발급 (보안 강화)
- ✅ 토큰은 개발팀이 안전하게 관리
- ❌ 토큰이나 비밀번호를 직접 만지지 말 것 — 개발팀이 관리
- ❌ 토큰을 Slack이나 이메일로 공유하지 말 것

MCP로 할 수 없는 것

회사 재무 시스템에 직접 접속
인사 시스템에서 급여 정보 조회
고객의 개인정보 대량 다운로드

이런 민감한 시스템은 MCP로 연결하지 않습니다.

자주 묻는 질문

Q: MCP 연결이 안 됐으면 어떻게 하나요? A: 개발팀에 "GitHub 연결 안 된 것 같아요. 확인해주세요"라고 말하세요.

Q: 새 서비스를 연결하고 싶어요. A: 개발팀에 "Slack 연결해줄 수 있나요?"라고 요청하세요.

Q: 내가 토큰을 직접 설정할 수 있나요? A: 아니요. 보안상 개발팀만 가능합니다. 자연어로 요청만 하세요.

Q: MCP가 갑자기 안 되면? A: 토큰이 만료됐을 수 있습니다. 개발팀에 "MCP 토큰 확인 부탁드립니다"라고 말하세요.

Q: 어떤 데이터를 조회할 수 있는지 어떻게 알아요? A: AI한테 "지금 DB에서 어떤 테이블을 볼 수 있어?"라고 물어보세요.

💡 실천 팁

- ✅ 필요한 데이터는 자연어로 요청

- ✅ GitHub PR은 AI에게 자동 생성 맡기기
 - ✅ 최신 정보는 웹 검색 활용
 - ✅ 라이브러리 문서는 AI한테 "공식 문서 확인해줘"
 - ✅ 새 서비스가 필요하면 개발팀에 편하게 요청
 - ❌ 토큰이나 비밀번호를 직접 만지지 말 것
 - ❌ 민감한 개인정보 조회는 자제
 - ❌ MCP 설정 파일을 직접 수정하지 말 것
-
-

다음 단계

08장 — 멀티에이전트에서 AI 여러 명을 동시에 일시키는 법을 배울 수 있습니다.

AI 여러 명이 동시에 일하게 하기 — 멀티에이전트

왜 필요한가요?

AI 한 명이 느리면, 여러 명한테 나눠서 시키는 것입니다.

예를 들어:

혼자: "구조 파악해줘 → 버그 찾아줘 → 문서 써줘" 순서대로 하나씩 (느림)

여러 명: A는 구조 파악, B는 버그 찾기, C는 문서 작성 → 동시에 진행 (빠름)

Claude Code에서 이렇게 일을 나눌 수 있습니다.

어떤 때 효과적인가요?

상황	혼자 할 때	멀티에이전트일 때
파일 3개 수정	순서대로 30분	동시에 10분
구조 분석 + 코드 수정	분석 끝날 때까지 대기	분석은 조수가, 나는 다른 작업
대규모 리팩토링	하루 종일	AI팀이 3시간

4가지 상황별 가이드

상황 1: 뭔가 조사해야 할 때 — 조수한테 맡기기

지금 하는 작업을 방해받지 않고, 별도로 조사나 문서 작성을 맡기고 싶을 때.

💬 AI한테 이렇게 말하세요:

"백그라운드에서 프로젝트의 모든 API 엔드포인트를 찾아서 API.md로 정리해줘"

"백그라운드에서 지난 3개월 변경사항 분석해서 CHANGELOG.md 만들어줘"

"백그라운드에서 이 프로젝트의 의존성 목록을 분석하고 업데이트가 필요한 것 정리해줘"

AI가 별도 조수(Subagent — AI가 따로 띄우는 보조 AI)를 만들어서 조사합니다.

- 당신은 다른 일을 계속할 수 있음
- 완료되면 결과 파일이 자동 생성됨
- 완료 알림을 받을 수 있음

💡 "백그라운드에서"라고 말하는 게 핵심입니다. 이 말이 없으면 AI가 직접 처리하느라 당신은 기다려야 합니다.

조수 결과 확인하기

💬 AI한테 이렇게 말하세요:

"아까 시킨 API 분석 끝났어?"

"백그라운드 작업 결과 보여줘"

조수가 만든 파일(예: API.md, CHANGELOG.md)을 AI가 보여줍니다.

상황 2: 여러 파일을 동시에 수정해야 할 때

2~5개 정도의 독립적인 파일을 한꺼번에 고치고 싶을 때.

💬 AI한테 이렇게 말하세요:

"이 3개 파일을 동시에 수정해줘: login.js는 UI 개선, auth.js는 에러 처리, theme.css는 다크모드"

"이 5개 컴포넌트 파일 동시에 TypeScript로 변환해줘"

AI가 각 파일마다 별도 작업 공간(worktree — Git이 만드는 임시 작업 폴더)을 만들어 동시에 수정하고, 자동으로 병합합니다.

⚠️ **핵심 규칙:** 같은 파일을 여러 AI가 동시에 수정하면 충돌이 납니다. 각각 다른 파일만 지정하세요.

조합

결과

❌ worker-1이 auth.js + worker-2도 auth.js

충돌! 수정이 덮어써짐

조합	결과
✅ worker-1은 auth.js + worker-2는 api.js	OK. 동시 진행, 자동 병합
✅ worker-1은 auth.js + worker-2는 styles.css + worker-3은 README.md	OK. 3배 빠름

상황 3: 프론트+백엔드+테스트를 동시에 해야 할 때 — 팀 구성

복잡한 프로젝트에서 설계, 개발, 테스트를 동시에 진행하고 싶을 때.

💬 AI한테 이렇게 말하세요:

"팀을 꾸려서 새 기능을 만들어줘. 설계자는 API 설계, 개발자는 코드 작성, 테스터는 테스트 작성"

"팀으로 결제 시스템 리팩토링해줘. 프론트 담당, 백엔드 담당, DB 마이그레이션 담당 나눠서"

AI들이 서로 소통하면서 진행합니다:

설계자가 API 설계 완료

개발자가 그걸 보고 코드 구현

테스터가 테스트 작성하고 검증 → 모두 완료되면 결과 보고

팀 작업 vs 병렬 작업 — 뭐가 다른가요?

구분	병렬 작업 (상황 2)	팀 작업 (상황 3)
소통	각자 독립적으로 진행	서로 결과를 주고받으며 진행
적합한 경우	독립적인 파일 수정	서로 연관된 작업
예시	3개 파일 각각 수정	설계 → 개발 → 테스트 순서

상황 4: 큰 작업을 통째로 맡기고 싶을 때

💬 AI한테 이렇게 말하세요:

"/ultrawork 이미지 로딩 속도 2배 빠르게 하고 빌드 시간 50% 줄여줘"

"/ultrawork 사용자 인증 시스템 전체 리팩토링해줘. OAuth 추가하고 보안 감사도 해줘"

AI가 계획 → 실행 → 검증까지 3단계를 자동으로 진행합니다. 자세한 내용은 04장 — 스킴과 커맨드를 참고하세요.

상황 선택 판단표

어떤 방식을 써야 할지 모르겠으면 이 표를 참고하세요.

상황	방식	이유
"이 폴더 분석해줘"	조수 한 명	한 명이면 충분, 빠름
"이 3개 파일 동시에 수정해줘"	병렬 작업	각각 독립적이니까 동시 처리
"앱 만들어줘" (설계+개발+테스트)	팀 구성	소통하며 순서대로 협업
"버그 원인 모르겠어. 여러 각도에서 분석해줘"	팀 구성 (3-5명)	다각도 분석
"이 큰 프로젝트 알아서 해줘"	/ultrawork	자동 계획+실행+검증

💡 잘 모르겠으면 그냥 AI한테 "이 작업을 가장 빠르게 하려면 어떻게 해야 해?"라고 물어보세요. AI가 적합한 방식을 추천합니다.

실전 예시

예시 1: 긴급 버그 수정 + 정기 작업

💬 AI한테 이렇게 말하세요:

"백그라운드에서 이번 주 변경사항 CHANGELOG.md로 정리해줘. 나는 긴급 버그 수정할게"

→ 조수가 CHANGELOG를 쓰는 동안, 당신은 버그 수정에 집중

예시 2: 대규모 파일 변환

💬 AI한테 이렇게 말하세요:

"src/components/ 안의 .js 파일 5개를 동시에 TypeScript로 변환해줘"

→ 5개 파일이 동시에 변환되어 5배 빠름

예시 3: 신기능 개발

💬 AI한테 이렇게 말하세요:

"팀으로 결제 기능 만들어줘. API 설계 → 프론트 구현 → 테스트 순서로"

→ AI팀이 순서대로 협업하며 완성

자주 묻는 질문

Q: 멀티에이전트를 쓰면 비용이 더 드나요? A: 네, AI 여러 명이 동시에 작업하니 비용이 늘어납니다. 하지만 시간이 절약되므로, 큰 작업에서는 효율적입니다.

Q: 조수가 실수하면 어떻게 하나요? A: 조수의 결과를 확인한 뒤, 문제가 있으면 AI한테 "이 부분 다시 해줘"라고 시키세요.

Q: 몇 명까지 동시에 작업할 수 있나요? A: 보통 3~5명이 적합합니다. 너무 많으면 관리가 어려워지고 충돌 위험이 커집니다.

Q: 멀티에이전트 vs 멀티세션, 뭐가 다른가요? A: 멀티에이전트는 하나의 프로젝트에서 여러 AI가 동시 작업, 멀티세션은 여러 프로젝트를 각각 관리하는 것입니다. 자세한 내용은 09장을 참고하세요.

💡 핵심 규칙 3가지

규칙 1: 같은 파일 금지

같은 파일을 여러 AI가 동시에 수정하면 충돌이 납니다. 각각 다른 파일만 지정하세요.

규칙 2: 조수는 "백그라운드에서"라고 말하기

"백그라운드에서"라는 말이 있어야 조수가 따로 일하고, 당신은 다른 일을 할 수 있습니다.

규칙 3: 큰 작업은 /ultrawork

계획부터 검증까지 AI가 자율적으로 진행합니다. 자세히 지시하지 않아도 됩니다.

💡 실천 팁

- ✅ 단순 조사는 조수 한 명에게 맡기기
 - ✅ 독립적인 파일 수정은 병렬 작업으로
 - ✅ 복잡한 작업은 팀 구성으로
 - ✅ 아주 큰 작업은 /ultrawork로
 - ❌ 같은 파일을 여러 AI가 동시에 수정하지 말 것
 - ❌ 5명 이상 동시 작업은 자제
-

다음 단계

09장 — 멀티세션 — 여러 프로젝트를 동시에 관리하는 법

10장 — AI 기억력 관리 — AI가 까먹지 않게 하는 법

여러 프로젝트를 동시에 관리하기 — 멀티세션

이게 뭔가요?

브라우저 탭을 여러 개 띄우듯, **AI 대화창을 여러 개 띄우는 것**입니다.

이런 상황일 때

프로젝트 A를 작업하다가 프로젝트 B에서 긴급 버그 발생

B를 빨리 고치고 다시 A로 돌아가고 싶음

두 프로젝트의 진행상황을 각각 유지하고 싶음

멀티세션 없이는 `/clear` 해야 해서 A 작업을 잃지만, 멀티세션이면 왔다갔다 하면서 둘 다 유지됩니다.

멀티세션 vs 멀티에이전트 — 뭐가 다른가요?

구분	멀티세션 (이 장)	멀티에이전트 (08장)
개념	AI 대화창을 여러 개 띄우기	하나의 대화에서 AI 여러 명 동시 작업
목적	프로젝트 전환	속도 향상
비유	브라우저 탭 여러 개	직원 여러 명

사용법

새 세션 만들기

💬 AI한테 이렇게 말하세요:

```
"새 세션 열어줘. 이름은 project-b"
```

또는 터미널에서:

```
claude --session project-b
```

기존 세션은 그대로 열려있고, 새 프로젝트용 세션이 따로 열립니다.

💡 세션 이름은 나중에 찾기 쉽게 의미있는 이름으로 지으세요.

세션 전환하기

💬 AI한테 이렇게 말하세요:

```
"project-a 세션으로 전환해줘"
```

또는 터미널에서:

```
claude --switch project-a
```

💡 전환하면 이전 세션의 대화는 그대로 남아있습니다. 다시 돌아오면 이어서 작업할 수 있습니다.

세션 목록 보기

현재 열려있는 세션 확인:

```
cmux list
```

결과 예시:

```
project-a    (활성 - 30분 전 마지막 활동)
project-b    (대기 - 2시간 전)
docs         (대기 - 어제)
```

세션 정리하기

작업 완료한 세션은 닫아서 정리:

💬 AI한테 이렇게 말하세요:

"이 세션 정리해줘. 더 이상 필요 없어"

오래된 세션을 한꺼번에 정리하려면:

💬 AI한테 이렇게 말하세요:

"7일 넘은 세션 전부 정리해줘"

실전 패턴

하루 일과 예시

아침 9시 — 오늘의 메인 프로젝트 시작:

```
claude --session settle-feature-auth
```

오전 10시 — 긴급 결제 버그 발생:

```
claude --session settle-fix-payment → 버그 수정 완료 → claude --switch  
settle-feature-auth 로 원래 작업 복귀
```

오후 2시 — 문서 작성 필요:

```
claude --session docs-api → API 문서 작성
```

퇴근 전 — 정리:

각 세션에서 "커밋해줘" → 세션 정리

세션 이름 규칙

일관된 이름을 쓰면 관리가 쉬워집니다.

규칙	예시	설명
[프로젝트] - [작업] - [기능]	settle-feature-auth	정산 프로젝트, 기능 추가, 인증

규칙	예시	설명
[프로젝트] - [작업] - [기능]	settle-fix-payment	정산 프로젝트, 버그 수정, 결제
[카테고리] - [주제]	docs-api	문서 카테고리, API 주제

✓ `settle-feature-auth`, `settle-fix-payment`, `docs-api`
✗ `aaa`, `test123`, `나중에하기` — 나중에 뭔지 모름

핸드폰에서도 AI에게 일을 시킬 수 있습니다

맥 미니나 서버에서 Claude Code를 실행 중이라면, 핸드폰에서도 접속할 수 있습니다.

설정하기

☞ AI한테 이렇게 말하세요:

"원격 접속 환경 세팅해줘. mosh로"

mosh(Mobile Shell — 와이파이가 바뀌어도 자동 재연결되는 원격 접속 도구)를 설정하면:

- 핸드폰 터미널 앱에서 AI에게 일을 시킬 수 있음
- WiFi가 끊겨도 자동으로 재연결
- 카페 → 지하철 → 회사 이동 중에도 끊기지 않음

모바일 추천 앱

기기	추천 앱
iPhone/iPad	Blink Shell (유료, 안정적) 또는 Termius
Android	Termux (무료) 또는 JuiceSSH

사용 예시

상황: 퇴근 후 지하철에서 긴급 배포 확인

핸드폰에서 Blink Shell 앱 실행

`mosh 내서버주소` 접속

`claude --resume latest` 입력

"배포 상태 확인해줘" 입력

AI가 확인 후 보고

⚠ 서버 쪽에도 mosh가 설치되어 있어야 합니다. 개발팀에 확인하세요.

⚠ 주의사항

같은 프로젝트를 두 세션에서 동시에 열지 마세요

❌ 세션 A에서 project-x, 세션 B에서도 project-x → 파일 충돌

✅ 세션 A는 project-x, 세션 B는 project-y

같은 프로젝트의 다른 기능을 작업하려면, 하나의 세션에서 순차적으로 진행하세요.

메모리는 세션마다 독립적

세션 A에서 한 대화는 세션 B에서 모릅니다.

정보를 공유하려면:

CLAUDE.md에 규칙 추가 → 모든 세션에서 자동으로 읽음

메모리에 저장 → 다른 세션에서도 확인 가능

자세한 내용은 10장을 참고하세요.

중요한 건 자주 커밋

💬 AI한테 이렇게 말하세요:

"지금까지 한 거 커밋해줘"

멀티세션에서 세션을 전환하기 전에 커밋해두면 안전합니다.

세션이 너무 많으면

세션을 10개 이상 열면 관리가 어려워집니다. 완료된 세션은 바로바로 정리하세요.

💡 실천 팁

- ✅ 프로젝트별로 세션 분리
- ✅ 세션 이름을 **[프로젝트] - [작업] - [기능]** 형식으로
- ✅ 전환 전에 반드시 커밋
- ✅ 완료된 세션은 바로 정리

- ✅ 핸드폰 접속은 mosh로 안정적으로
 - ❌ 같은 프로젝트를 여러 세션에서 동시에 열지 말 것
 - ❌ 세션을 10개 이상 유지하지 말 것
-
-

다음 단계

10장 — AI 기억력 관리 — 멀티세션에서 특히 중요한 기억력 관리법

AI가 자꾸 까먹을 때 — 기억력 관리법

왜 까먹나요?

AI의 기억력에는 한계가 있습니다. 대화가 길어지면 앞부분을 잊어버립니다.

시험공부할 때 책 뒷부분을 읽다 보니 앞부분을 까먹는 것과 같습니다.

기억력 상태 확인

`/status` 를 입력하면 현재 기억력 사용량을 확인할 수 있습니다.

기억력	상태	증상	권장 조치
0-50%	여유	빠르고 정확함	계속 진행
50-75%	주의	약간 느려질 수 있음	길어지지 않도록 주의
75-90%	위험	느려지고, 앞부분을 잊기 시작	<code>/compact</code> 실행
90%+	긴급	매우 느리고, 심하면 에러 발생	<code>/compact</code> 또는 <code>/clear</code> 즉시 실행

`/compact` 와 `/clear` 에 대한 설명은 00장을 참고하세요.

기억력 아끼는 법 3가지

1. 한 번에 한 가지만 (가장 중요)

이것만 지키면 기억력 문제의 80%를 예방할 수 있습니다.

❌ "이 에러 고쳐주고, 기능도 추가하고, 문서도 써줘" → AI가 동시에 처리하려다 헛갈리고 기억력 낭비

✅ "먼저 이 에러 고쳐줘" → 끝나면 → "다음으로 기능 추가해줘" → 기억력 50% 절약, 품질 향상

왜 한 번에 한 가지가 좋은가요?

방식	기억력 사용	품질	속도
3가지 동시	90%	중간 (혼란)	느림
1가지씩 순서대로	30%씩	높음 (집중)	빠름

2. 새 주제 시작하면 /clear

한 기능 완료, 새 파일 작업 시작, 버그 수정 끝났을 때 → `/clear`

AI의 기억이 100%로 복구되어 다음 작업에 집중할 수 있습니다.

/clear를 써야 하는 신호

"방금 전 작업이랑 완전히 다른 주제를 시작할 때"

"AI 답변이 갑자기 느려졌을 때"

"AI가 내가 한 말을 까먹기 시작할 때"

"하나의 기능을 완성하고 다음 기능으로 넘어갈 때"

💡 `/clear` 전에 중요한 진행상황은 커밋해주세요. 대화 기록이 전부 사라집니다.

3. 큰 조사는 조수한테 맡기기

❌ "지난 100개 커밋을 분석해줘" → 기억력 바닥남

✅ "백그라운드에서 지난 100개 커밋 분석해서 CHANGELOG.md 만들어줘" → 조수(Subagent)가 대신 처리, 내 기억력은 아낌

기억력을 많이 쓰는 작업 vs 적게 쓰는 작업

기억력 많이 씬	기억력 적게 씬
"전체 프로젝트 분석해줘"	"이 파일 하나 분석해줘"
"100개 파일 전부 읽어줘"	"이 함수만 설명해줘"
"지난 1년 커밋 분석"	"최근 5개 커밋만 보여줘"

💡 범위를 좁힐수록 기억력을 아킵니다.

자세한 내용은 08장 — 멀티에이전트를 참고하세요.

AI한테 기억시키기

업무 규칙 가르치기 — CLAUDE.md

"모든 AI가 읽어야 하는 규칙"을 적어두는 파일입니다. 세션이 바뀌어도 AI가 매번 자동으로 읽습니다.

💬 AI한테 이렇게 말하세요:

"업무 규칙 추가해줘. 코드는 영어, 주석은 한국어, API 응답은 JSON 형식, console.log는 프로덕션에 남기지 말 것"

AI가 프로젝트의 CLAUDE.md 파일에 규칙을 추가합니다.

CLAUDE.md에 뭘 적으면 좋을까요?

카테고리	예시
코딩 스타일	"변수명은 camelCase, 함수명도 camelCase"
언어 규칙	"코드는 영어, 주석은 한국어"
금지사항	"console.log를 프로덕션 코드에 남기지 말 것"
API 형식	"API 응답은 {code, message, data} 형식"
테스트	"새 기능마다 테스트 코드 필수"

효과: 매번 설명하지 않아도 일관된 스타일 유지. 기억력 20% 절약.

규칙이 제대로 적용되는지 확인

💬 AI한테 이렇게 말하세요:

"지금 CLAUDE.md에 어떤 규칙이 있어?"

진행상황 기록하기 — 메모리

여러 세션에 걸쳐 프로젝트를 진행할 때, AI가 기억해야 할 정보를 메모리에 저장합니다.

💬 AI한테 이렇게 말하세요:

"지금까지 진행상황을 메모리에 저장해줘"

다음 세션에서:

"메모리에 저장된 진행상황 확인해줘"

AI가 자동으로 메모리를 읽고 이전 작업을 이어갑니다.

메모리 vs CLAUDE.md — 뭐가 다른가요?

구분	CLAUDE.MD	메모리
내용	변하지 않는 규칙	변하는 진행상황
예시	"코드는 영어로"	"로그인 기능 80% 완료"
수명	프로젝트 내내	작업 완료까지
읽는 시점	매 세션 시작할 때 자동	필요할 때 요청

효과: 멀티세션에서도 일관성 유지. 기억력 30% 절약.

⚠ 메모리에 비밀번호, API 키 같은 보안 정보를 저장하지 마세요.

💡 하루 업무 루틴

아침 — 프로젝트 시작:

"어제 작업한 로그인 기능 테스트해줘" (메모리에서 자동으로 진행상황을 불러옴)

오전 — 기능 A 작업:

"이메일 인증 기능 만들어줘"

기능 A 완료 후:

"커밋해줘" → `/clear` → "이번엔 비밀번호 리셋 기능 만들어야 해"

오후 — 기능 B 작업 중 기억력 80%:

`/compact`

퇴근 전:

"지금까지 진행상황 메모리에 저장하고 커밋해줘"

자주 묻는 질문

Q: CLAUDE.md를 직접 수정해도 되나요? A: AI한테 시키는 게 더 편하지만, 직접 수정해도 됩니다. 다음 세션부터 적용됩니다.

Q: 메모리에 저장한 내용은 어디에 있나요? A: AI한테 "메모리 내용 보여줘"라고 하면 확인할 수 있습니다.

Q: /compact와 /clear 중 뭘 써야 할지 모르겠어요. A: 같은 작업을 계속하면 `/compact`, 다른 작업으로 넘어가면 `/clear`. 헷갈리면 `/compact` 가 안전합니다.

Q: 기억력이 부족하면 AI가 에러를 내나요? A: 에러보다는 느려지고 앞부분을 잊어버립니다. `/status` 로 확인하고 미리 정리하세요.

💡 효율적인 대화법

한 번에 한 가지만

❌ "이 버그 고쳐주고, 문서 써주고, 테스트도 돌려줘"

✅ "이 버그 고쳐줘"

구체적으로 말하기

❌ "뭔가 이상해"

✅ "로그인이 안 되는데, 에러 메시지는 'TypeError: undefined is not a function'"

범위 좁히기

❌ "프로젝트 전체 분석해줘"

✅ "src/auth/ 폴더만 분석해줘"

기억력 아끼려면 조수 활용

❌ "지난 1년 커밋 전부 분석해줘"

✅ "백그라운드에서 지난 1년 커밋 분석해서 통계 만들어줘"

다음 단계

11장 — 자동 배포 — CI/CD 설정하기

troubleshooting — 문제 해결 가이드

코드를 자동으로 배포하기 — CI/CD

CI/CD가 뭔가요?

CI/CD(지속적 통합/배포)란 코드를 고치면 자동으로 서버에 반영되는 것입니다.

구글독스처럼 수정하면 자동 저장되듯, 코드를 업로드하면 자동으로 검사하고 배포합니다.

코드 수정 → GitHub에 업로드 → 자동 검사(버그 없나?) → 자동 배포(서버에 반영) → 끝!

왜 필요한가요?

구분	수동 배포	CI/CD
시간	30분 이상	5분
실수 가능성	높음 (사람이 하니까)	거의 없음 (자동이니까)
과정	서버 접속 → 파일 교체 → 재시작 → 테스트	GitHub에 업로드 → 자동 처리
결과	"아, 서버 설정 빠뜨렸다"	"자동으로 다 됐습니다"

CI/CD 없을 때 vs 있을 때

CI/CD 없을 때:

코드를 수정한다
개발자가 서버에 접속한다
파일을 하나하나 교체한다
서버를 재시작한다
브라우저에서 확인한다
"아, 환경 변수 빠뜨렸다" → 다시 2번부터...

CI/CD 있을 때:

코드를 수정한다

GitHub에 업로드한다 (커밋 + 푸시)

☕ 커피 마시면서 기다린다

알림: "배포 완료!" 또는 "배포 실패: OO 에러"

AI한테 설정 맡기기

새 프로젝트에 CI/CD 설정하기

💬 AI한테 이렇게 말하세요:

"/cicd-setup 이 프로젝트에 자동 배포 설정해줘"

AI가 자동으로 처리하는 것:

GitHub Actions 설정 — 자동화 규칙 파일 생성

테스트 자동 실행 — 코드 업로드할 때마다 테스트

서버 배포 설정 — 테스트 통과하면 자동 배포

배포 가이드 문서 — 팀이 참고할 수 있는 문서

더 구체적으로 시키기

💬 AI한테 이렇게 말하세요:

"/cicd-setup React + Node.js 프로젝트. GitHub 푸시하면 자동 테스트하고 통과하면 자동 배포해 줘. Slack 알림도 보내줘"

"/cicd-setup 이 프로젝트 Vercel에 자동 배포되게 설정해줘. main 브랜치에 머지되면 프로덕션 배포"

"/cicd-setup Docker 기반 프로젝트. AWS ECS에 자동 배포. 스테이징 환경도 만들어줘"

💡 프로젝트 기술 스택과 배포 대상을 알려줄수록 AI가 더 정확하게 설정합니다.

배포 확인하기

GitHub에서 확인

GitHub 프로젝트 페이지 → **Actions** 탭을 클릭하면 배포 상태를 볼 수 있습니다.

표시	의미	조치
✅ 초록색 체크마크	배포 성공	없음 — 정상!
❌ 빨간색 X 마크	배포 실패	AI한테 에러 메시지 보내기
🟡 노란색 원	배포 진행 중	잠시 기다리기

AI한테 물어보기

💬 AI한테 이렇게 말하세요:

"GitHub Actions 상태 확인해줘"

AI가 현재 배포 진행 상황, 성공/실패 여부, 소요 시간을 알려줍니다.

배포가 실패했을 때

에러 메시지 복사 → AI한테 보내기

💬 AI한테 이렇게 말하세요:

"GitHub Actions에서 배포가 실패했어. 이 에러 확인해줘: [에러 메시지 붙여넣기]"

AI가 자동으로:

- 에러 원인을 분석
- 어디를 고쳐야 하는지 설명
- 코드를 수정
- 다시 배포

자주 나오는 에러와 해결법

에러	원인	AI한테 이렇게
"npm test failed"	테스트 실패	"테스트 에러 고쳐줘"

에러	원인	AI한테 이렇게
"build failed"	빌드 에러	"빌드 에러 확인해줘"
"permission denied"	배포 권한 없음	개발팀에 "배포 권한 확인해주세요"
"timeout"	시간 초과	"배포 시간이 부족한 것 같아. 타임아웃 늘려줘"

💡 배포가 실패해도 **이전 버전이 자동 복구**됩니다. 사용자에게는 영향이 없으니 당황하지 마세요.

배포 환경 이해하기

스테이징 vs 프로덕션

환경	설명	누가 보나
스테이징	테스트용 서버 (가짜 데이터)	팀 내부만
프로덕션	실제 서버 (진짜 데이터)	고객

💬 AI한테 이렇게 말하세요:

"스테이징 환경에 먼저 배포해줘. 확인하고 프로덕션에 올릴게"

스테이징에서 먼저 확인 → 문제 없으면 프로덕션에 배포. 이렇게 하면 안전합니다.

배포 알림 받기

💬 AI한테 이렇게 말하세요:

"배포 성공/실패 알림을 Slack으로 보내게 설정해줘"

배포 상태를 매번 GitHub에서 확인하지 않아도 Slack에서 바로 알 수 있습니다.

⚠️ 주의사항

배포 권한 관리

✅ "팀장만 프로덕션에 배포할 수 있게 해줘"

✅ "스테이징은 모든 팀원이 배포할 수 있게 해줘"

❌ "아무나 프로덕션에 배포할 수 있게 해줘"

회사 규정에 따라 개발팀과 상의하세요.

비밀 정보 관리

✅ API 키는 GitHub Secrets에 저장 (코드에 직접 쓰지 않음)

✅ 데이터베이스 비밀번호는 환경 변수로 관리

❌ 코드 안에 `API_KEY="sk_live_xxx"` 같은 걸 넣지 말 것

❌ `.env` 파일을 GitHub에 업로드하지 말 것

💬 AI한테 이렇게 말하세요:

"API 키를 GitHub Secrets에 안전하게 설정해줘"

배포 전 테스트

CI/CD가 자동으로 테스트를 돌리지만, 로컬에서도 먼저 확인하면 더 안전합니다.

💬 AI한테 이렇게 말하세요:

"배포 전에 테스트 한 번 돌려줘"

"빌드가 정상적으로 되는지 확인해줘"

자주 묻는 질문

Q: CI/CD 설정을 내가 직접 해야 하나요? A: 아니요. `/cicd-setup` 스크립트를 쓰면 AI가 알아서 합니다.

Q: 배포가 실패하면 사이트가 멈추나요? A: 아니요. 이전 버전이 자동으로 유지됩니다. 새 배포가 성공할 때까지 기존 버전이 계속 서비스됩니다.

Q: GitHub Actions는 무료인가요? A: 공개 저장소는 무료입니다. 비공개 저장소는 월 2,000분까지 무료이고, 보통 충분합니다. 초과하면 요금이 발생할 수 있으니 개발팀에 확인하세요.

Q: 배포를 되돌릴 수 있나요? A: 네. AI한테 "이전 버전으로 롤백해줘"라고 하세요. AI가 안전하게 이전 버전으로 되돌립니다.

💡 실천 팁

- ✅ `/cicd-setup` 으로 한 번에 설정
 - ✅ GitHub Actions 탭에서 배포 상태 확인
 - ✅ 실패하면 AI한테 에러 메시지 붙여서 물어보기
 - ✅ 스테이징에서 먼저 확인 → 프로덕션에 배포
 - ✅ 배포 알림은 Slack으로 받기
 - ❌ 비밀 정보를 코드에 직접 넣지 말 것
 - ❌ 배포 권한을 무분별하게 열지 말 것
 - ❌ `.env` 파일을 GitHub에 올리지 말 것
-

다음 단계

troubleshooting — 배포 외 다른 문제 해결

08장 — 멀티에이전트 — 복잡한 배포는 AI팀으로 처리

이럴 때 이렇게 하세요 — 문제 해결 가이드

Q1. AI가 너무 느려요

원인

기억력이 가득 찼거나 대화가 너무 길어짐.

해결법

💬 AI한테 이렇게 말하세요:

`"/compact"` (대화를 요약해서 기억력 정리. 진행상황 유지)

`"/clear"` (모든 대화 삭제. 기억력 100% 복구)

`/compact` 와 `/clear` 상세 설명은 00장을 참고하세요.

⚠️ `/clear` 전에 커밋해주세요. 대화 기록이 전부 사라집니다.

그래도 느리면?

`/status` 로 기억력 확인 → 90% 이상이면 `/clear` 필수

Claude Code를 완전히 종료하고 다시 시작

네트워크 상태 확인 (Wi-Fi가 불안정하면 느려질 수 있음)

Q2. AI가 내 말을 못 알아들어요

원인

너무 추상적으로 설명함.

해결법

❌ "이 부분 좀 이상한데 봐줘"

✅ 구체적으로:

💬 AI한테 이렇게 말하세요:

"src/auth/login.js 45번 줄에서 Cannot read properties of undefined 에러가 나"

핵심: 파일 경로 + 에러 메시지를 함께 알려주세요.

잘 전달하는 공식

[어디서] + [뭐가 안 됨] + [에러 메시지]

예시:

"src/auth/login.js에서 + 로그인 안 됨 + TypeError: undefined"

"결제 페이지에서 + 구매 버튼이 안 눌림 + 콘솔에 에러 없음"

Q3. AI가 파일을 못 찾아요

원인

프로젝트 폴더에서 실행하지 않았거나, 경로가 잘못됨.

해결법

💬 AI한테 이렇게 말하세요:

"지금 어느 폴더에 있어? 파일 목록 보여줘"

프로젝트 폴더가 아니라면, 터미널에서 프로젝트 폴더로 이동 후 다시 시작:

```
cd ~/내_프로젝트_폴더
claude
```

💡 어떤 폴더에서 시작해야 하는지 모르겠으면 개발팀에 물어보세요.

Q4. AI가 권한을 계속 물어봐요

원인

default 모드(매번 승인 요청)로 설정됨.

해결법

💬 AI한테 이렇게 말하세요:

"모드를 bypassPermissions로 바꿔줘"

모드 설명은 03장을 참고하세요.

Q5. 브라우저 자동화가 안 돼요

원인

Chrome 프로필 설정 문제이거나 Playwright가 설치되지 않았을 수 있음.

해결법

💬 AI한테 이렇게 말하세요:

"브라우저 자동화가 안 되는데 환경 설정 확인해줘"

AI가 설정을 진단하고 자동으로 고칩니다.

그래도 안 되면:

"Playwright 다시 설치해줘"

자세한 내용은 06장을 참고하세요.

Q6. 외부 도구(DB, GitHub) 연결이 안 돼요

원인

MCP(외부 도구 연결) 설정이 안 됨.

해결법

개발팀에 말하세요:

"GitHub MCP 설정을 활성화해줄 수 있나요?"

MCP 설명은 07장을 참고하세요.

💡 MCP 연결은 개발팀만 할 수 있습니다. 자연어로 요청만 하세요.

Q7. 갑자기 연결이 끊겼어요

원인

네트워크 문제 또는 세션 타임아웃.

해결법

💬 AI한테 이렇게 말하세요 (재접속 후):

"방금 연결 끊겼어. 이전 세션 이어가줘"

```
claude --resume latest
```

자주 끊기면?

mosh를 설정하면 와이파이가 바뀌어도 자동 재연결됩니다.

💬 AI한테 이렇게 말하세요:

"mosh 설치하고 원격 접속 설정해줘"

02장을 참고하세요.

Q8. 실수로 중요한 파일을 바꿨어요

해결법

💬 AI한테 이렇게 말하세요:

"방금 한 변경 다 되돌려줘"

AI가 자동으로 이전 상태로 복구합니다.

Git으로 커밋했다면 더 확실하게 되돌릴 수 있습니다:

💬 AI한테 이렇게 말하세요:

"마지막 커밋 되돌려줘"

💡 이래서 자주 커밋하는 게 중요합니다. 커밋해두면 언제든지 되돌릴 수 있습니다.

Q9. AI가 같은 실수를 반복해요

원인

AI가 당신의 규칙을 모름. 매 세션마다 기억이 초기화되기 때문.

해결법

💬 AI한테 이렇게 말하세요:

"업무 규칙 추가해줘: API 응답은 항상 {code, message, data} 형식으로. console.log는 프로덕션에 남기지 말 것"

AI가 CLAUDE.md에 규칙을 추가합니다. 이후 모든 세션에서 자동으로 따릅니다. 자세한 내용은 10장을 참고하세요.

Q10. 어디에 문제가 있는지 모르겠어요

해결법

💬 AI한테 이렇게 말하세요:

"이 기능이 안 돼. 뭘 했는지, 뭐가 안 되는지, 에러 메시지를 알려줄 테니 진단해줘"

AI가 체계적으로 확인합니다:

- 프론트엔드 (화면에 보이는 부분)
- 백엔드 (서버에서 처리하는 부분)
- 데이터베이스 (데이터 저장소)
- 네트워크 (인터넷 연결)

용어집

처음 보는 용어가 있다면 여기서 찾아보세요.

용어	설명
터미널	컴퓨터에 글자로 명령하는 검은 화면
커밋(commit)	코드 변경사항을 저장하는 것. 구글독스의 "버전 기록 만들기"와 비슷
PR (Pull Request)	"이 코드 검토해주세요"라는 팀 공유 요청
브랜치(branch)	코드를 따로 분리해서 작업하는 공간. 원본에 영향 없이 실험 가능
MCP	AI가 외부 서비스(GitHub, DB 등)에 접근하는 통로. USB 포트 같은 것
모드	AI의 동작 방식. default(매번 승인), plan(분석만), bypassPermissions(자동)
토큰	AI의 "기억력 단위". 대화가 길면 토큰을 많이 씀. 한국어 1글자 = 약 1-2토큰
CI/CD	코드를 올리면 자동으로 테스트하고 서버에 반영하는 시스템
Subagent	AI가 별도로 띄우는 조수. 백그라운드에서 작업 수행
CLAUDE.md	AI에게 업무 규칙을 알려주는 파일. 매 세션마다 자동으로 읽음

비용 안내

Claude Code는 사용한 만큼 비용이 발생합니다.

비용 구조

- 토큰(token): AI가 읽고 쓰는 텍스트의 단위
- 한국어 1글자는 약 1-2토큰
- 코드는 보통 1줄에 10-30토큰

비용 확인

`/status` 를 입력하면 현재 세션의 비용이 표시됩니다.

비용 절약 팁

방법	효과
한 번에 한 가지만 시키기	기억력 절약 → 비용 절감
<code>/compact</code> 로 기억력 정리	불필요한 토큰 사용 방지
구체적으로 지시하기	재시도 횟수 감소
대용량 조사는 조수에게 위임	메인 세션 비용 절감 (08장 참고)
새 주제면 <code>/clear</code>	불필요한 기억 제거

❌ 절대 하면 안 되는 것

금지 행위	왜 위험한가	대신 이렇게
"강제로 푸시해줘" (force push)	팀의 코드가 날아감	"새 커밋으로 수정해줘"
비밀번호/API 키를 코드에 직접 쓰기	유출 위험	"API 키 안전하게 설정해줘"
DB에서 대량 삭제/구조 변경	데이터 복구 불가	개발팀에 요청
같은 파일을 여러 AI가 동시에 수정	충돌 발생	각각 다른 파일만 지정
실제 사용자 계정으로 브라우저 테스트	개인정보 위험	테스트 계정 사용
메모리/CLAUDE.md에 비밀번호 저장	유출 위험	.env 파일에만 저장

💡 AI도 파괴적 작업은 먼저 확인을 물어봅니다. "이걸 하면 데이터가 삭제됩니다. 진행할까요?" 같은 안전장치가 있습니다.

도움 요청 방법

계속 해결이 안 되면 개발팀에 이렇게 알려주세요:

제목: [Claude Code] ○○ 문제

- 하려던 일: [구체적으로]
- 현재 상황: [에러 메시지 또는 스크린샷]
- 이미 시도한 것: [한 것들 나열]
- 재현 방법: [같은 문제를 다시 만드는 방법]

개발팀 Slack에 @here Claude Code 문제 있습니다 로 빠르게 알릴 수 있습니다.

💡 예방 팁

- ✅ 자주 커밋하기 — 문제 생기면 바로 이전으로 복구 가능
- ✅ `/status` 로 상태 확인 — 기억력, 비용, 모드 확인
- ✅ 한 번에 한 가지만 — 문제 원인 파악이 쉬움
- ✅ 느려지면 `/compact` — 기억력 정리
- ✅ 새 주제면 `/clear` — 깔끔하게 시작
- ✅ 에러 메시지는 그대로 복사해서 AI한테 보내기

빠른 참조표

상황	해결법	참고
AI가 느려	<code>/compact</code> 또는 <code>/clear</code>	00장
AI가 까먹어	<code>/status</code> 확인, CLAUDE.md 업데이트	10장
파일 못 찾아	프로젝트 폴더에서 cLaude 재시작	-
권한 계속 물어봐	"모드를 bypassPermissions로 바꿔줘"	03장

상황	해결법	참고
연결 끊김	<code>claude --resume latest</code>	02장
파일 되돌리기	"방금 한 변경 되돌려줘"	05장
외부 도구 안 됨	개발팀에 MCP 설정 요청	07장
브라우저 안 됨	"환경 설정 확인해줘"	06장
같은 실수 반복	CLAUDE.md에 규칙 추가	10장